Private Stablecoin on Ethereum

PLabs

2026 年 10 月 9 日

摘要

本文提出 Ethereum 上的隐私稳定币**发行协议**：任何人可部署由选定储备 ERC-20 按 1:1 背书的 `ReserveVault` 与 `NotePool`。以 USDC 为储备的实例记为 **pUSDC**；持有形式是 pERC20 票据，不替代底层储备。用户入金后直接持有私人票据，不必先持有公开储备代币；持有与转账对链隐藏。流通量由合约保证与储备 1:1，不靠发行方披露。

表 1: USDC 与以 USDC 为储备的协议实例 pUSDC（pUSDC 列高亮）

维度	USDC	pUSDC
入金即隐私	公开地址余额	**私人票据；用户不必有公开地址**
持有人 / 余额 / 转账图	公开	**对链隐藏**
铸出瞬间	可花可兑债权	库存；不可花、不可兑
流通 ↔ 储备	发行方披露与审计	**合约强制 1:1，链上可核对**
赎回	经发行方	**持有人自证；无需许可**
阻断资金	发行方可冻结	仅合规冻结
披露	账本全公开	选择性；查看密钥

设计有两个分离。**付款人 ≠ 收款人**：锁储备代币的是入金平台、desk 或任意第三方，收票据的是用户。**铸造 ≠ 成为货币**：预铸票据归协议地址，不能转、不能兑；同交易付入等额储备之后，才变成可花、可兑的流通债权。一般地，硬不变量是储备代币余额 $\geq$ `circulating`；在 pUSDC 实例中即 `ReserveVault.USDC` $\geq$ `circulating`。法币出入金仍信任出入金平台。链上合约不可变，不信任运营方、平台或 relayer。唯一的链上特权是合规冻结。

关键词：隐私稳定币；发行协议；无需许可部署；Ethereum；零知识证明；pERC20；入金即隐私；无信任发行；无信任锚定

目录

1 引言 **3**

1.1 问题 . . . 3
1.2 核心思想 . . . 3
1.3 贡献 . . . 3
1.4 本文组织 . . . 4

2 相关工作 **4**

2.1 Ethereum 上的屏蔽池：先有公开币，再变隐私 . . . 4
2.2 非 Ethereum 链上的隐私稳定币 . . . 4
2.3 对照与 Ethereum 约束 . . . 4

3 系统概览 **5**

3.1 架构 . . . 5
3.2 两类票据 . . . 6
3.3 信任与目标 . . . 6

4 协议 **7**

4.1 票据、action 与签名 . . . 7
4.2 入口与授权 . . . 7
4.3 电路扩展：CP 与 CT . . . 8
4.4 Π_{buy}：购买（入金主路径） . . . 8
4.5 Π_{transfer} 与 Π_{redeem} . . . 9
4.6 职责矩阵 . . . 9
4.7 Private DeFi 构建 . . . 10
4.8 与公开 DeFi 的组合 . . . 10

5 安全与隐私 **10**

5.1 命题与攻击矩阵 . . . 12

6 合规与可审计性 **13**

7 讨论与局限 **14**

8 结论 **14**

1 引言

1.1 问题

稳定币已经是链上金融的结算层。USDC、USDT 以及 DAI、PYUSD 等品种承担报价、抵押、支付和 RWA 申购赎回。它们能当结算层，是因为它们是 Ethereum 上的公开资产：和任意代币组成交易对，被现有合约直接持有。

公开账本带来两个问题：

1. **账本透明**。余额和转账谁都能看。金库、做市商、基金的头寸、流向和对手方可以按图索骥；工资和供应商结算同样不宜公开。

2. **发行即债权**。稳定币一经铸出就能转、能兑。储备在发行方和银行里，流通和储备是否匹配，靠披露和审计。用户信的是发行方，不是可以核对的合约状态。

隐私稳定币要让持有人、金额和对手方离开公开账本，但按需仍能向监管或审计披露。它还应当像今天的稳定币一样被持有：用户交法币，得到的就是这枚币，而不是先持有一笔公开 USDC，再送进池里藏起来。这枚币随后要能和公开资产交易，留在 Ethereum 的 DeFi 里。发行和赎回也不应再依赖对发行方的信任。把隐私迁到另一条链，等于放弃这套结算层（第 2 节）。

1.2 核心思想

1. **付款人 $\neq$ 收款人**。锁 USDC 的人和收票据的人不是同一个。用户向平台提供收款地址 pa_U；平台生成输出票据和承诺 $\mathsf{cmx}_{\mathsf{out}}$，把票据明文交还用户，并签 `BuyAuth`。链上出现的是付款人地址和金额，不是用户。入金本就信任平台，承诺由谁算出不影响链上绑定：签进 `BuyAuth` 的那个 $\mathsf{cmx}_{\mathsf{out}}$ 拿到全部付款。

2. **铸造 $\neq$ 成为货币**。发行方铸出的票据必须归公开的协议地址 pa_P，称为协议 note（库存）。电路公开进出 pa_P 的价值，合约据此禁止库存参与 `transfer`。协议 note 只能经 `buy`（同交易 USDC 入库）变成用户 note（流通），或经 `redeem`（同交易 USDC 出库）变回库存。背书发生在出售的那一笔交易里，不发生在铸造时。

1.3 贡献

1. **无需许可的发行协议**。任何人可部署 `ReserveVault` 与 `NotePool`，选定任一 ERC-20 作 1:1 储备；pUSDC 是以 USDC 为储备的运行实例。

2. **入金即隐私**。付款人锁储备代币，用户链上的第一笔债权已经是私人票据——不是先持有公开储备再屏蔽。之后的转账由用户本地证明、relayer 提交，用户 EOA 不必出场。

3. **无信任发行、锚定与赎回**。预铸不可转移。只有付入等额储备才增加流通。储备余额 $\geq$ `circulating`（pUSDC 中为 `ReserveVault.USDC` $\geq$ `circulating`）由合约维护。持有人自证即可兑出，中介不能改向，运营方不能阻断。

4. **留在 Ethereum 的 DeFi 里**。协议下发行的私人票据是 Ethereum 上的资产。与公开协议的交换可以压进一笔原子交易，链上看不到持有人。

与公开 USDC 的对照见表 1（以 pUSDC 实例为例）。

1.4 本文组织

第 2 节谈相关工作。第 3 节给架构与目标。第 4 节描述协议、主路径与 DeFi 组合。第 5、6 节谈安全、隐私、命题与合规冻结。第 7 节谈局限。

2 相关工作

2.1 Ethereum 上的屏蔽池：先有公开币，再变隐私

公开稳定币能当结算层，是因为它是 Ethereum 上的一等资产。隐私稳定币要扮演同一个角色，入金和流通都得像今天的稳定币一样——**入金即隐私：用户链上的第一笔债权已经是这枚私人币**，随后它还能留在 Ethereum 的 DeFi 里，和公开资产交易，同时持有与支付对链隐藏。

Tornado Cash、Railgun、Aztec、Privacy Pools [7, 8, 9] 做不到入金这一步。它们要求用户**已经持有**公开稳定币，再把余额送进池里变成隐私形式。今天的入金不是这样：法币进来，用户直接持有 USDC。屏蔽池给不出这条路径。私人美元只能是公开美元的后处理。

因为池里装的是「已经被屏蔽的公开币」，和 DeFi 的组合只能先出池、在公开面做完、再视需要进池。池也不发行美元，不保证流通和储备 1:1。

本协议（以 pUSDC 为例）的入金与此对应：付款人付出储备代币，平台按用户的收款地址生成私人票据，用户直接持有。协议下发行的私人票据留在 Ethereum 上，之后仍可与公开或隐私域内流动性交互（第 4.7、4.8 节）。

2.2 非 Ethereum 链上的隐私稳定币

Aleo 上已经有隐私稳定币 [10, 11, 12, 13]。主网上的 USDCx 以 Circle xReserve 锁在 Ethereum 上的原生 USDC 为 1:1 储备；USAD 由 Paxos Labs 以 USDG 为储备发行。铸币由 `MINTER_ROLE` 一类角色控制。隐私只在 Aleo 上成立；桥回 Ethereum 就不再隐私。和 Uniswap、Aave 的交易变成跨链铸销。Aleo 由持有铸币角色的发行方映射储备；本协议任何人可部署 `ReserveVault/NotePool`，把未付款库存和已付款流通分成两类票据，流通量由合约约束（pUSDC 以 USDC 为储备）。

2.3 对照与 Ethereum 约束

表 2 汇总三类方案。隐私稳定币必须建在 Ethereum 上：组对、借贷、托管都在这里。

表 2: 相关工作对照（末列为可部署发行协议，以 USDC 实例 pUSDC 说明）

	屏蔽池	Aleo	本协议（pUSDC）
入金	先持有公开币再屏蔽	跨链存 / 角色铸币	**法币 → 私人票据**
结算层	Ethereum	Aleo（桥回则公开）	**Ethereum**
流通 ↔ 储备	不维护锚定	发行方 / 桥侧映射	**合约强制 1:1**
铸币门控	不适用	`MINTER_ROLE`	库存不可转；付储备才流通
公开 DeFi	先出池再交互	跨链	**同链原子组合**

USDC 的流通与储备匹配靠发行方披露；发行方可对地址加黑名单 [4]。以 USDC 为储备的实例 pUSDC 继承该信用与风险，但流通侧 1:1 由合约强制。合规冻结是本协议唯一链上特权，不能

把储备转走。

3 系统概览

3.1 架构

图 1 按层给出组件清单；消息流见图 2。琥珀为库存，青为流通，蓝为 Vault。



图 1: 发行协议分层架构（无箭头；pUSDC 为 USDC 储备实例）。按层列出组件；消息流见图 2。

参与方与合约角色如下（任何人可部署一对 `ReserveVault`/`NotePool`）。

- **用户**：持 ask 与票据明文。入金时只提供 pa_U；之后本地选票据、出证明、签名。长期 ask 不交出。
- **出入金平台**：持储备代币（pUSDC 中为 USDC）。入金时生成 $\mathsf{cmx}_{\mathsf{out}}$、签 `BuyAuth`、交还明文；出金时收储备，付法币。
- **运营方**：持 ask_P，`mint` 备货，构造购买 bundle；持查看密钥副本只读审计。
- **Relayer**：提交已签 bundle；无花费权；任何地址可任。
- **合规管理方**：更新冻结根；部署后不可更换。
- **ReserveVault / NotePool**：Vault 持选定储备 ERC-20；池维护承诺树、nullifier、冻结根。池的 `buy/redeem` 只接受来自 Vault 合约的调用（划款与改账同笔原子）；Vault 上对应入口公开，任何人可调，无需运营方许可。公开的 `ReserveVault.redeem` 无需许可。

合约不可升级，也不能改验证密钥、费率或提交名单。树深建议 32；过半叶子后进入只出模式，只接受赎回，保证未花费票可整张兑出。

3.2 两类票据

表 3: 两类票据（流通列高亮）

	协议 note（库存）	用户 note（流通）
所有者	pa_P（公开）	用户 pa（仅在承诺中）
花费密钥	ask_P	用户 ask
`transfer`	禁止	允许
USDC 背书	无	购买时锁入

池上三个量：`totalSupply`（总量）为全部票据面值；`uncirculated`（未流通）为归 pa_P 的协议 note 面值；`circulating`（已流通）为用户 note 面值，满足

$$\texttt{circulating} = \texttt{totalSupply} - \texttt{uncirculated}.$$

库存可先于储备存在：`mint` 只增加 `totalSupply` 与 `uncirculated`，不移动 USDC。ask_P 泄露不会让库存变成可兑流通，但可能让库存无法再出售。

3.3 信任与目标

下文用三类标签：**A** = 假设（Assumptions），**G** = 安全目标（Goals），**P** = 隐私目标。后文按编号引用。

假设 A（链下）。

- **A6**（链下信任）用户信任选用的平台与运营方：平台按 pa_U 出票并签 `BuyAuth`；出金后付法币。平台若把 $\mathsf{cmx}_{\mathsf{out}}$ 做成自己的票，等价于入金不入账。A6 不参与链上论证。

假设 A（链上）。

- **A1** 证明系统可靠（正确性与知识可靠性）。
- **A2** 承诺隐藏且绑定；签名不可伪造。
- **A3** 底层 USDC 可冻结 Vault 地址（继承 Circle 黑名单）。
- **A4** 以太坊共识与单笔交易原子性成立。
- **A5** 用户妥善保管自己的 ask。
- **A7** 合规管理方可更新冻结根（可覆盖票据），但不能转移储备。

在 A1–A5 与 A7 下，G1–G6 对任意行为的运营方、平台与 relayer 成立。

安全目标 G。

- **G1** $\texttt{USDC} \geq \texttt{circulating}$（储备覆盖流通）。
- **G2** 库存不能经 `transfer` 流出。
- **G3** 花费用户 note 须持有对应 ask。

- **G4** buy 的 USDC 只从 BuyAuth 签名者划出，且 $\mathsf{cmx}_{\mathsf{out}}$ 得全部 a。
- **G5** redeem 只流向持有人签入的地址或计划，金额为 $a - \varphi$。
- **G6** 赎回不依赖运营方签名或白名单；除合规冻结外无人能阻止或拖延（提交走公开的 Vault）。

隐私目标 P。

- **P1** 入金不必有公开地址。
- **P2** transfer 不泄漏双方地址与金额，不与用户 EOA 绑定。
- **P3** 隐私针对公开链；运营方持查看密钥是设计如此。

4 协议

4.1 票据、action 与签名

pERC20 [3] 沿用 Zcash Orchard 票据模型 [1]。票据明文 $\mathsf{Note} = (g_d, pk_d, v, \rho, \psi, \mathsf{rcm})$。承诺隐藏且绑定；花费时公开 nullifier。新票据的 ρ 等于同 action 被花费票据的 nullifier。

一个 action 花一张旧票、出一张新票；若干 action 组成 bundle。记 bundle 内被花费票据面值之和为 V_{in}、新出票据面值之和为 V_{out}，则公开字段

$$\texttt{valueBalance} \;=\; V_{\text{in}} - V_{\text{out}}$$

是本笔交易的票据净值，由电路与 binding 签名约束，并写入 SIGHASH。符号约定：valueBalance <0 表示本笔在票据侧创造价值（如 mint）；>0 表示销毁价值（如 burn）；$=0$ 表示票据内部守恒（buy/redeem/transfer）。它与池上全局计数 totalSupply 不同：后者是合约状态，在入口成功后按规则增减；前者只描述这一笔 bundle。

全部签名签同一摘要 SIGHASH（覆盖 nullifier、承诺、密文、valueBalance、recipientMeta、executor）。无 ask 与 binding 密钥则改不了摘要。executor $\neq 0$ 时仅该地址可提交。

4.2 入口与授权

每个入口把允许的 valueBalance 与 executor 固定写入验证：调用方不能自选净值，否则可在不付 USDC 时把库存变成流通。表 4 给出各入口的取值；例如 mint(a) 只出面值 a、不花旧票，故 valueBalance $= -a$，随后合约执行 totalSupply $+= a$、uncirculated $+= a$。

表 4: 池入口（主路径 buy/redeem 高亮；「仅 Vault」指池侧 msg.sender）

入口（池）	池侧 msg.sender	valueBalance	要点
mint(a)	issuer	$-a$	只增库存；无 USDC
burn(a)	issuer	$+a$	销毁库存
buy$(a, \mathsf{cmx}_{\mathsf{out}})$	仅 ReserveVault	0	库存净减 a；同交易划入 a USDC
redeem(a)	仅 ReserveVault	0	库存净增 a；付 $a - \varphi$
transfer	任意	0	库存进出均为 0

「仅 ReserveVault」不是特权角色：对外入口是公开的 ReserveVault.buy/redeem；bundle 的 executor 亦固定为 Vault，防止绕过划款直调池。

谁出资产，谁签去向——与入口 `buy`/`redeem` 成对。二者算法不同。

`BuyAuth`（USDC→note）。 付款人对 EIP-712 结构体签名：

$$\texttt{BuyAuth} = \mathsf{Sign}_{\mathsf{payer}}\big(\mathsf{EIP712}\{\mathsf{payer},\, a,\, \mathsf{cmx}_{\mathsf{out}},\, \mathsf{nonce},\, \mathsf{deadline}\}\big).$$

EOA 用 **secp256k1 ECDSA**（链上 `ecrecover`）；合约钱包用 **ERC-1271**。

主路径上平台自生成 $\mathsf{cmx}_{\mathsf{out}}$ 再签。Vault 的 `buy` 验签与 `deadline`，标记 $(\mathsf{payer}, \mathsf{nonce})$ 已用，再划入 a USDC；电路 CT 保证 $\mathsf{cmx}_{\mathsf{out}}$ 得全部 a。

`RedeemAuth`（note→USDC）。 不是第二份 EIP-712，也无单独签名算法。持有人把收款摘要写入 bundle：

$$\mathsf{redeemDigest} = H(\mathsf{DOMAIN}_{\mathsf{REDEEM}}, \mathsf{recipient}, a),$$
$$\mathsf{recipientMeta} = H(\texttt{ReserveVault}, \mathsf{redeemDigest}).$$

再以 note 的 **SpendAuth**（Orchard 花费授权，随机化公钥 rk）与 **binding signature** 签覆盖 $\mathsf{recipientMeta}$ 的同一 $\mathsf{SIGHASH}$。调用 `redeem` 时 $(\mathsf{recipient}, a)$ 作为公开参数传入——赎回出池，金额与收款地址本就公开。合约 `ReserveVault` 用 `address(this)` 与自己即将转账的 $(\mathsf{recipient}, a)$ 重算 $\mathsf{recipientMeta}$，与 bundle 中已签字段比对；relayer 改地址或金额则摘要不符、验签失败。Vault 地址写入哈希，防止跨合约重放。

4.3 电路扩展：CP 与 CT

C1–C6 不区分库存与用户票。`ActionPusdc` 增加：

CP（库存进出）。

$$\mathsf{isP}_{\mathsf{in}} = (\text{旧票主人} = \mathsf{pa}_P), \quad vP_{\mathrm{in}} = \mathsf{isP}_{\mathsf{in}} \cdot v_{\mathrm{old}},$$
$$\mathsf{isP}_{\mathsf{out}} = (\text{新票主人} = \mathsf{pa}_P), \quad vP_{\mathrm{out}} = \mathsf{isP}_{\mathsf{out}} \cdot v_{\mathrm{new}}.$$

合约加总 $\texttt{protocolIn} = \sum vP_{\mathrm{in}}$、$\texttt{protocolOut} = \sum vP_{\mathrm{out}}$。

CT（目标输出）。

$$\mathsf{isT} = (\mathsf{cmx}_{\mathrm{new}} = \mathsf{cmx}_T), \quad \mathsf{isT} \cdot (v_{\mathrm{new}} - v_T) = 0, \quad \mathsf{isT} \cdot \mathsf{isP}_{\mathsf{out}} = 0.$$

`buy` 时 $(\mathsf{cmx}_T, v_T) = (\mathsf{cmx}_{\mathsf{out}}, a)$，使付款全部进入指定票。

4.4 Π_{buy}：购买（入金主路径）

备货 `mint`：issuer 向 pa_P 输出金额 a 的协议 note；USDC 与 `circulating` 不变。图 2 是入金主路径；蓝色区域为单笔交易内原子步骤。

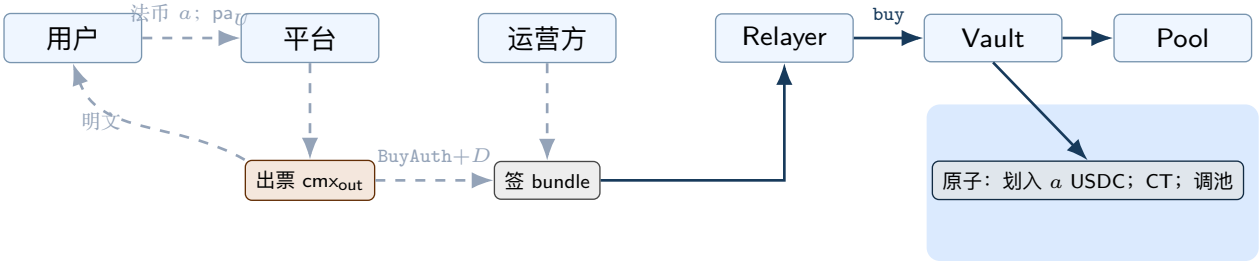


图 2: Π_{buy}：平台出票并签 BuyAuth；运营方签库存；Vault 原子入库（蓝底）。

1. 平台出票：生成一次性 dummy 与 $\mathsf{Note}(\mathsf{pa}_U, a)$（以 $\rho = \mathsf{nf}_D$ 固定唯一性种子），得 $\mathsf{cmx}_{\mathsf{out}}$，明文交还用户。
2. 平台签 BuyAuth。
3. 运营方花费库存、配上 dummy，证明并签名；另花库存做找零。
4. Vault 验签、划 USDC、调池；任一步失败则回滚。

链上可见：付款人、a、$\mathsf{cmx}_{\mathsf{out}}$、nullifier。不可见：用户 pa、用户 EOA。

4.5 Π_{transfer} 与 Π_{redeem}

图 3 把转账与赎回画在一起。青底为隐私转账原子步，蓝底为赎回原子步。

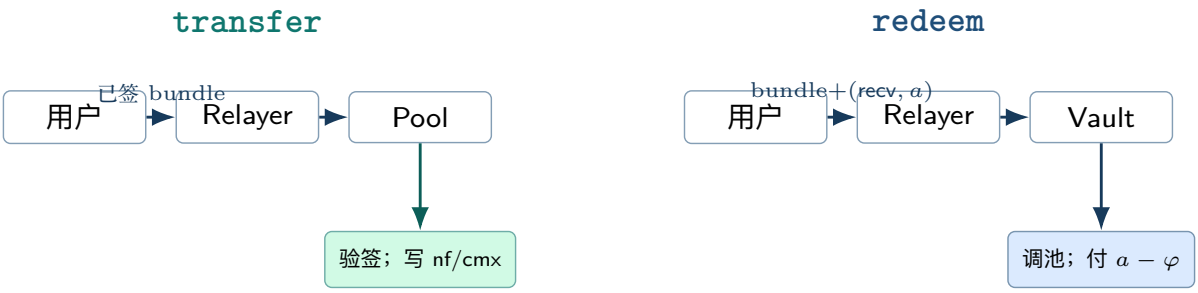


图 3: Π_{transfer}（左，青）与 Π_{redeem}（右，蓝）。色块为单笔交易内原子步骤。

transfer：executor $= 0$ 时任人可提交。Relayer 报酬为同 bundle 内小费 note。redeem：输出协议 note 是收款，不需 ask_P；持有人或任意 relayer 调公开的 ReserveVault.redeem，Vault 再调池。法币出金时 recipient 为出金平台。

4.6 职责矩阵

表 5: 各流程职责（buy 行高亮）

流程	平台	运营方	用户	Relayer
mint	—	构造并提交	—	—
buy	出票；签 BuyAuth	签库存	给 pa_U	提交
transfer	—	—	选票；签名	提交
redeem	出金时收 USDC	—	签订单	提交
DeFi	—	反向预签 buy	签计划	提交

4.7 Private DeFi 构建

隐私稳定币既可以经 `unshield` 融入现有公开流动性（第 4.8 节），也可以直接构建与现有流动性成交的应用，不必先赎回再操作。后者按对手方资产形态分两种：

1. **隐私稳定币 ↔ 公开资产**：在一步交易里直接构造成交场景，隐私腿与公开腿同笔结算，而不是先 `redeem`/`unshield` 再进公开协议。
2. **隐私稳定币 ↔ 已屏蔽的公开资产**：双方均为隐私票据（例如原生隐私资产与经 shield 进入池的稳定币），撮合与结算全程留在隐私域。

PLabs 给出了方式 2 的产品原型：**PEX**（链上隐私交易所）。当前交易对为 **P20/sUSDC**：P20 是 pERC20 协议发行的原生隐私资产；sUSDC 是 USDC 经 shield 进入隐私池后的票据。二者以非托管、基于 CLOB 的方式成交。体验见 `https://app.plabs.online/pex`。

方式 2 的撮合与结算骨架可以很容易扩展到方式 1：把一侧换成公开资产腿，在同笔交易中直接对接公开余额或公开池，其余非托管约束保持不变。这与第 4.8 节的「赎回 → 公开操作 → 回隐私」路径彼此独立。

4.8 与公开 DeFi 的组合

与第 4.7 节的一步成交不同：此处先把隐私稳定币兑回公开 ERC-20，再进入 Uniswap、Aave 等现有协议，必要时再 `buy` 回隐私面。公开协议只认 ERC-20。持有人把「赎回 → 公开操作 → 回隐私」压进一笔交易。换回 pUSDC 是一次 `buy`：用户给 $\mathtt{pa}_U$，运营方出票并预签 `buyCall`。

组件含 `redeemForDefi`、防重入协调器、白名单 Adapter 与目标隐私池；`planDigest` 与赎回订单域标签互不相交。性质：原子；计划绑定；Adapter 不转发任意调用；残差只能到公开地址；夹击损失受滑点约束。



正向：pUSDC→pWETH　　反向：pWETH→pUSDC

图 4: 与公开 DeFi 的正反向。蓝/青为链上原子执行；琥珀为库存相关的预签 `buy`。

5 安全与隐私

在 A1–A5、A7 与不可变部署下讨论安全目标。

每个合法入口不改变 $\mathtt{slack} = \mathtt{USDC} - \mathtt{circulating} - \mathtt{feesAccrued}$；入口末尾断言 slack 不变差。故 G1 成立。库存不能经 `transfer` 流出（G2）。`BuyAuth`+CT 钉死 $\mathtt{cmx}_{\mathtt{out}}$（G4）。赎回本地证明、无需运营方（G6）。

表 6: 公开可观测量（「不可见」列高亮隐私边界）

流程	链上可见	链上不可见
`buy`	付款人、a、$\mathsf{cmx}_{\mathsf{out}}$、nullifier	用户 pa、EOA
`transfer`	nullifier、输出承诺、relayer	双方 pa、金额、EOA
`redeem`	recipient、a、nullifier	被花费票的 pa 与原金额
聚合	`circulating`、库存、Vault 余额	单用户余额

三条泄漏。(1) 少见金额的 buy/redeem 时间相关。(2) 付款人地址公开，但不是用户。(3) 平台与运营方串通可把 KYC 对上 pa——对合规的让步。

5.1 命题与攻击矩阵

在 A1–A5、A7 与不可变部署下，对任意行为的运营方、平台与 relayer，下列命题成立。记 $\mathtt{slack} = \mathtt{USDC} - \mathtt{circulating} - \mathtt{feesAccrued}$。

命题 1（I1）. 任意时刻 $\mathtt{circulating} = \mathtt{totalSupply} - \mathtt{uncirculated}$：二者由合约对进出 pa_P 的价值加减维护，等式按定义成立。

命题 2（I2′）. 每个合法入口使 slack 恰好不变，且入口末尾断言 slack 不恶化。故在 $\mathtt{slack} \geq 0$ 时任何入口不能打破 $\mathtt{USDC} \geq \mathtt{circulating} + \mathtt{feesAccrued}$；已有亏空（A3）时入口不得加深亏空，但仍允许赎回。

命题 3（G1）. 唯一使 circulating 增加的入口是 buy，且同交易等额 USDC 入库、CT 钉死 $\mathsf{cmx}_{\mathsf{out}}$ 得全部 a。故无付款不能增发可兑流通。

命题 4（G2）. transfer 要求 $\mathtt{protocolIn} = \mathtt{protocolOut} = 0$。库存不能经转账流出为用户 note。

推论 1. mint 超发库存只增 uncirculated，不增可兑债权；资金安全不依赖铸币额度。树满风险由只出模式消解（第 3 节）。

命题 5. 泄露 ask_P 不能在不付 USDC 时把库存变成可兑流通（命题 3、4）。持有者至多能自付 USDC 购买；可用性上可能重写库存密文使运营方无法再售，故 ask_P 宜门限保管。

命题 6（G3）. 花费用户 note 须对应 ask 的 SpendAuth。查看密钥（含运营方所持 FVK）不含 ask，不能花费、不能改已签 SIGHASH。

命题 7（G4）. buy 只从通过验签的 BuyAuth.payer 划走 a；$(\mathsf{payer}, \mathsf{nonce})$ 单次有效；CT 强制输出含金额恰为 a、且非 pa_P 的 $\mathsf{cmx}_{\mathsf{out}}$。付款人不能被换成他人，收款票不能被截留或替换。

命题 8（G5）. redeem/redeemForDefi 的收款地址（或计划）由持有人写入已签字段；Vault（或协调器）用即将执行的公开参数重算摘要并比对。改地址、金额或计划则验签失败。

命题 9（G6）. 赎回证明本地生成，不需运营方签名。对外入口 ReserveVault.redeem 公开：任何 relayer（或收款地址自行）可提交；池侧 onlyReserveVault 只约束「必须经 Vault 兑付」，不是许可名单。除合规冻结（A7）外无人能阻止或拖延；冻结不能转移储备。

表 7: 攻击矩阵

#	攻击	对策	依据
S-1	库存当流通转走	`transfer` 禁库存进出	命题 4
S-2	无付款增发流通	固定 `valueBalance`；同交易入库	命题 2,3
S-3	谎报进出协议价值	CP 由电路计算	A1
S-4	伪造 pa_P	链上核对池常量	—
S-5	替换赎回去向	Vault 重算摘要	命题 8
S-6	审查转账/赎回	入口无需许可	命题 9
S-7	法币不交付	链下信任	A6
S-8	无限 `mint` 填树	只出模式	推论 1
S-9	付款买成自身票/截留	`BuyAuth`+CT	命题 7
S-10	`BuyAuth` 重放	nonce + deadline	命题 7
S-11	篡改已签 bundle	SIGHASH 全覆盖	C3,C5
S-12	以 FVK 花费	ask 不在 FVK 中	命题 6
S-13	双花	nullifier 集	C2
S-14	花费冻结票据	C6	§6
S-15	抢先花掉 dummy	仅作废该笔购买	Π_{buy}
S-16	夹击 DeFi swap	滑点上下限	§4.8
S-17	手续费扣押赎回	φ 部署常量	表 4
S-18	向协调器转入代币	期末清扫余额	§4.8
S-19	嵌套调用 ERC-1271	防重入	§4.8
S-20	换验证密钥/升级	无此类权限	§3
S-21	国库拉黑阻赎回	赎回不转国库	表 4
S-22	亏空后全部赎回回滚	slack 不恶化断言	§5
S-23	树满无法赎回	只出模式预留叶子	§3
S-24	ask_P 泄露重写库存	无资金影响；门限签名	命题 5

运营方与 relayer 合谋也不能无付款增发或改资金去向。运营方与合规管理方可冻票，转不走储备。与 USDC 发行方合谋超出模型（A3）。

6 合规与可审计性

合规管理方维护冻结树根。花费须证明承诺不在冻结集合中。冻结按 cmx；价值仍计入 `circulating`；不能转移储备。合约不验证根的正确性，范围由管理方决定。运营方所持查看密钥只读。`uncirculated`、`circulating` 与 Vault 余额链上可对。

表 8: 三种票据相关结构

结构	问题	证明	有效根
承诺树	是否存在	成员（C1）	任一历史根
nullifier 集	是否已花	合约查表（C2）	—
冻结树	是否冻结	非成员（C6）	仅当前根

冻结树为索引 Merkle 树 [6]。畸形根可停全部花费；不能转移储备。

7 讨论与局限

底层 USDC 风险继承自发行方。法币与库存出售无链上托管。不可变使缺陷只能迁池。自付入金时入口退化为普通屏蔽。开放问题：Adapter 白名单、冻结治理、其它隐私池的退出入口规格。

8 结论

本文给出 Ethereum 上的隐私稳定币发行协议：任何人可部署 `ReserveVault` 与 `NotePool`，由选定储备 ERC-20 按 1:1 背书。以 USDC 为储备的实例记为 pUSDC；持有形式是 pERC20 票据。库存与流通由所有者区分；付等额储备才流通；储备余额 $\geq$ `circulating` 可核对。入金后用户直接持有私人票据。谁出钱谁定去向。屏蔽池要求先有公开币再屏蔽；协议下发行的私人票据应像稳定币一样被持有，并留在 Ethereum 的 DeFi 里。

参考文献

[1] Zcash Protocol Specification, Orchard.

[2] EIP-20; EIP-712; EIP-1271.

[3] Cyimon. Add ERC: Private Fungible Tokens (pERC20). ethereum/ERCs #1817, 2026.

[4] Circle, USDC on-chain supply and blacklist documentation.

[5] J. Groth. On the Size of Pairing-Based Non-interactive Arguments. EUROCRYPT 2016.

[6] Aztec Labs. Indexed Merkle Tree.

[7] A. Pertsev et al. Tornado Cash Privacy Solution. 2019.

[8] Railgun Project. Railgun Privacy System. https://docs.railgun.org.

[9] V. Buterin et al. Blockchain Privacy and Regulatory Compliance. 2023.

[10] Y. Fanusie, V.-L. Jaber, M. Green. Stablecoin Privacy. Aleo, 2026.

[11] Aleo. Private Stablecoins. https://docs.aleo.org/build/common-uses/private_stablecoins/.

[12] Circle. USDCx on Aleo Testnet via Circle xReserve. 2025.

[13] Aleo Network Foundation and Paxos Labs. USAD. 2025.